

Experiment No.	6
Experiment Title	Bagging, Boosting, and Stacked Ensemble Models ¹
Student Name	Sanghamithra Namboodiri
Register Number	3122247001055
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	30th August 2026

1 Objective

- To understand ensemble learning strategies: Bagging, Boosting, and Stacking.
- To implement Bagging (Decision Tree base estimator) and Boosting (AdaBoost, Gradient Boosting) classifiers.
- To build a Stacked Ensemble model combining SVM, Naïve Bayes, and Decision Tree base learners with a Logistic Regression meta-learner.
- To compare ensemble models in terms of accuracy, stability, and generalization.
- To analyze the effect of each ensemble strategy on the bias–variance trade-off.

2 Problem Statement

Given 30 numeric cell-nucleus features computed from a digitized image of a breast mass fine-needle aspirate, the task is to classify the mass as **malignant** or **benign** (binary classification). **Input:** a 30-dimensional numeric feature vector per sample (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension, each reported as a mean, standard error, and "worst" value). **Output:** a binary label, **diagnosis** $\in \{0 = \text{benign}, 1 = \text{malignant}\}$. **Objective:** train and tune Bagging, Boosting, and Stacking classifiers on this task and compare their predictive performance and bias–variance behaviour.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository
Number of Samples	569 (raw); 346 after IQR outlier removal
Number of Features	30 numeric (after dropping the id column)
Number of Classes	2 (0 = Benign, 1 = Malignant)
Missing Values	None (0 across all 32 raw columns)
Train-Test Split	80/20 stratified $\Rightarrow$ 276 train / 70 test rows

4 Exploratory Data Analysis

Dataset Overview

The raw dataset has shape 569×32 (`id`, `diagnosis`, and 30 numeric features). After dropping `id`, mapping `diagnosis` to $\{0, 1\}$, and applying IQR-based outlier removal, the modelling dataset has shape 346×31 .

Statistical Summary

Table 2: Representative feature statistics (raw data, $n = 569$)

Feature	Mean	Std	Min	Max
<code>radius_mean</code>	14.13	3.52	6.98	28.11
<code>area_mean</code>	654.89	351.91	143.50	2501.00
<code>concavity_worst</code>	0.272	0.209	0.000	1.252

Inference: `area_mean` shows very high relative variability ($\text{std} \approx 54\%$ of the mean) compared to `radius_mean` ($\text{std} \approx 25\%$ of the mean), consistent with area scaling quadratically with radius and therefore amplifying measurement spread. `concavity_worst` ranges from 0 up to 1.25, indicating some tumours have essentially no concave regions while others are heavily indented – this feature is a strong candidate discriminator between benign and malignant masses, which is reflected in it being one of the standard WDBC features used for diagnosis.

Missing Value Analysis

All 32 raw columns reported 0 missing values, both before and after cleaning. No imputation was necessary for this dataset.

Class Distribution

Table 3: Class distribution before and after outlier removal

Stage	Benign (0)	Malignant (1)	Total
Raw dataset	357	212	569
After IQR outlier removal	263	83	346

Inference: the raw dataset is already moderately imbalanced (63% benign / 37% malignant), but outlier removal makes this considerably worse – malignant cases drop by 60.8% versus only 26.3% for benign cases, leaving a final ratio of roughly 76%/24%. This is a strong signal that IQR filtering is disproportionately removing malignant samples (plausibly because malignant tumours naturally produce more extreme cell-nucleus measurements), and it foreshadows the lower recall values observed later for some models (Section 10).

Correlation Matrix

Not generated by the current script. `perform_eda` did not produce a correlation heatmap for this run – the only figure the script generates is the ROC curve (Section 10). If a correlation matrix is required for submission, it would need to be added separately, e.g. via `sns.heatmap(X.corr())` on the 30 numeric features.

Expected pattern (not yet verified): strong positive correlation is expected within the "size" feature family – `radius_mean`, `perimeter_mean`, and `area_mean` are geometrically dependent measurements of the same tumour shape and should form a visibly correlated block, along with their corresponding `_worst` counterparts. This has a direct modelling implication: tree-based ensembles (Bagging, AdaBoost, Gradient Boosting) handle correlated features gracefully by splitting on whichever is locally most informative, whereas a Logistic Regression model would see inflated/unstable coefficients if fed these correlated features directly – not a concern here, since the Stacking meta-learner only sees the three base learners' predictions, not the raw correlated features.

Feature Distribution

Not generated by the current script. No per-feature distribution plots were produced by this run; the only figure output is the ROC curve (Section 10).

Indirect evidence of skew (from the outlier-flagging counts): `area_se` had the most flagged outliers of any non-identifier feature (65 of 569), followed by `area_worst` (35) and `radius_se/perimeter_se` (38 each) – all "standard error" or "worst" variants of size-related measurements. This suggests these columns are right-skewed with a long tail of large values (consistent with a subset of unusually large or irregular tumours), which is what one would expect for a mixed benign/malignant medical measurement dataset. A histogram or KDE plot per feature would confirm this directly if added to the script.

5 Data Preprocessing

- **Missing value handling:** not required – no missing values were present in any column.
- **Encoding:** the target `diagnosis` was mapped from `{B, M}` to `{0, 1}`. No categorical feature encoding was needed since all 30 modelling features are numeric.
- **Outlier removal:** IQR-based filtering was applied per numeric column, reducing the dataset from 569 to 346 rows (see Section 4 for the class-imbalance side effect this introduced).
- **Identifier removal:** the non-predictive `id` column was dropped before modelling.
- **Feature scaling:** applied selectively, not globally. Bagging, AdaBoost, and Gradient Boosting use decision-tree base learners, which are scale-invariant and therefore do *not* require standardization. Within the Stacked Ensemble, only the SVM and Logistic Regression meta-learner pipelines apply `StandardScaler` internally, since those two algorithms are distance/coefficient-magnitude sensitive; Gaussian Naïve Bayes and the Decision Tree base learner do not need scaling.

- **Train-test split:** 80/20 stratified split (`random_state=42`), preserving the benign/malignant ratio in both the 276-row training set and the 70-row test set.
- **Cross-validation:** 5-fold StratifiedKfold (shuffled, `random_state=42`) was used for every hyperparameter search, so every fold preserves the overall class ratio.
- **Feature engineering:** none was applied; all 30 original numeric features were used directly.

6 Implementation (Code)

Key excerpts of the implementation are shown below; the full script (`exp7.py`) performs EDA, preprocessing, model training, hyperparameter tuning, and evaluation end-to-end.

```

1  # Bagging: Decision Tree base estimator, tuned over
2  # n_estimators / max_samples / max_features
3  bagging = BaggingClassifier(
4      estimator=DecisionTreeClassifier(random_state=42),
5      random_state=42
6  )
7  bagging_params = {
8      "n_estimators": [3, 5, 10, 20],
9      "max_samples": [0.5, 0.8, 1.0],
10     "max_features": [0.5, 0.8, 1.0]
11 }
12 bagging_grid = GridSearchCV(bagging, bagging_params, cv=cv,
13                             scoring="accuracy", n_jobs=-1)
14 bagging_grid.fit(X_train, y_train)
15
16 # AdaBoost: depth-1 decision stump base estimator
17 adaboost = AdaBoostClassifier(
18     estimator=DecisionTreeClassifier(max_depth=1, random_state
19                                     =42),
20     random_state=42
21 )
22 adaboost_params = {
23     "n_estimators": [50, 100, 150],
24     "learning_rate": [0.01, 0.1, 0.2, 0.5]
25 }
26
27 # Gradient Boosting
28 gradient_boosting = GradientBoostingClassifier(random_state=42)
29 gradient_params = {
30     "n_estimators": [50, 100, 150],
31     "learning_rate": [0.01, 0.1, 0.2],
32     "max_depth": [1, 2, 3]
33 }
34
35 # Stacked Ensemble: SVM + Naive Bayes + Decision Tree -> Logistic
36 Regression
37 base_models = [

```

```

36     ("svm", Pipeline([("scaler", StandardScaler()),
37                       ("svm", SVC(kernel="rbf", C=1.0,
                                   random_state=42))])),
38     ("nb", GaussianNB()),
39     ("dt", DecisionTreeClassifier(max_depth=3, random_state=42))
40 ]
41 meta_learner = Pipeline([("scaler", StandardScaler()),
42                           ("logistic", LogisticRegression(
43                               max_iter=1000,
44                               random_state=42)
45                           )])
46 stacking = StackingClassifier(estimators=base_models,
                                final_estimator=meta_learner,
                                cv=5, n_jobs=-1)

```

7 Machine Learning Algorithm

Bagging (Bootstrap Aggregation)

Working principle: train B independent copies of a base learner on B bootstrap re-samples of the training set (sampling n instances with replacement), then aggregate their predictions. **Mathematical formulation:** for bootstrap samples $D_1, \dots, D_B$ drawn from training set D of size n , train $h_b = \text{DecisionTree}(D_b)$ for $b = 1, \dots, B$; the ensemble prediction is the majority vote,

$$H(x) = \text{mode}_{b=1, \dots, B} h_b(x).$$

Advantages: reduces variance, fully parallelizable across estimators, robust to overfitting of individual high-variance base learners (deep trees). **Limitations:** does not reduce bias – if the base learner is systematically biased, bagging does not fix it; requires many estimators before variance-reduction benefits saturate; less interpretable than a single decision tree.

AdaBoost

Working principle: sequentially train weak learners (here, depth-1 decision stumps), up-weighting misclassified samples after each round so later learners focus on the hardest cases, then combine all learners via a weighted vote. **Mathematical formulation:** initialize weights $w_i = 1/n$; for $m = 1, \dots, M$, fit weak learner h_m minimizing the weighted error $\epsilon_m = \sum_i w_i \mathbb{1}[h_m(x_i) \neq y_i]$, compute its weight $\alpha_m = \frac{1}{2} \ln\left(\frac{1-\epsilon_m}{\epsilon_m}\right)$, and update $w_i \leftarrow w_i \exp(-\alpha_m y_i h_m(x_i))$ (then renormalize); the final prediction is $H(x) = \text{sign}\left(\sum_{m=1}^M \alpha_m h_m(x)\right)$. **Advantages:** directly reduces bias, simple and effective with weak learners, resistant to overfitting in many practical settings. **Limitations:** sensitive to noisy data and outliers (misclassified noisy points get repeatedly up-weighted), inherently sequential and therefore not parallelizable, sensitive to the choice of weak learner.

Gradient Boosting

Working principle: build an additive model stage-wise, where each new tree is fit to the negative gradient (pseudo-residuals) of a differentiable loss function with respect to the current ensemble's predictions. **Mathematical formulation:** initialize $F_0(x) =$

$$\arg \min_c \sum_i L(y_i, c); \text{ for } m = 1, \dots, M, \text{ compute pseudo-residuals } r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}},$$

fit a tree h_m to $\{r_{im}\}$, find the optimal step size γ_m , and update $F_m(x) = F_{m-1}(x) + \nu \gamma_m h_m(x)$ where ν is the learning rate. **Advantages:** often the most accurate of the tree-based ensembles, supports flexible loss functions, can address both bias and variance with proper tuning. **Limitations:** sequential training is slow, many hyperparameters to tune, prone to overfitting with too many estimators or too high a learning rate, less interpretable than a single tree.

Stacked Ensemble

Working principle: train several diverse (heterogeneous) base learners on the training data, then use their out-of-fold predictions as engineered features to train a meta-learner that combines them optimally. **Mathematical formulation:** given base learners $h_1, \dots, h_K$ trained on D , generate meta-features $z_i = (h_1(x_i), \dots, h_K(x_i))$ via internal k -fold cross-validation (to avoid leakage), then train a meta-learner g on $\{(z_i, y_i)\}$; the final prediction is $g(h_1(x), \dots, h_K(x))$. **Advantages:** combines complementary strengths of different model families, often outperforms any single constituent model, the meta-learner choice is flexible. **Limitations:** computationally expensive (every base learner is refit across CV folds plus a final fit), higher risk of overfitting the meta-learner if cross-validation is not handled carefully, and harder to interpret or deploy than a single model.

8 Complexity Analysis

Let n = training set size (276), d = number of features (30), B = number of Bagging estimators (up to 20), M = number of boosting rounds (up to 150), and K_{cv} = number of internal stacking CV folds (5).

Table 4: Theoretical time complexity of each ensemble method

Model	Training	Prediction (per sample)	
Bagging	$O(B \cdot n \log n \cdot d)$	$O(B \cdot d)$	Yes
AdaBoost	$O(M \cdot n \cdot d)$ (stumps)	$O(M)$	No (s)
Gradient Boosting	$O(M \cdot n \log n \cdot d)$ (deeper trees)	$O(M)$	No (se)
Stacking	$O(K_{cv} \sum_k \text{Cost}(h_k)) + \text{Cost}(\text{meta})$	$\sum_k \text{Cost}(h_k) + \text{Cost}(\text{meta})$	Partially (base)

Notes: Bagging's per-tree cost benefits from being fully independent across estimators, so it parallelizes best of the four. AdaBoost's stumps (depth 1) are cheap individually but must be trained strictly in sequence. Gradient Boosting is the most expensive tree-based method here since it both trains sequentially *and* uses deeper trees (up to depth 3). Stacking's true cost is dominated by its most expensive base learner – the RBF-kernel

SVM, whose training cost can approach $O(n^2)$ – $O(n^3)$ in the worst case – multiplied by the $K_{cv} = 5$ internal refits needed to generate leak-free meta-features. At this dataset’s scale ($n = 276$, $d = 30$), all four methods remain fast in absolute terms, but the current script does not instrument wall-clock training time (unlike Experiments 3 and 4, which used `time.time()`); adding that instrumentation would allow this theoretical comparison to be validated empirically.

9 Experimental Setup

Python Version	<i>(fill in via <code>python --version</code>)</i>
Scikit-Learn Version	<i>(fill in via <code>python -c "import sklearn; print(sklearn.__version__)"</code>)</i>
IDE	
Operating System	
Hardware	

10 Hyperparameter Selection

Table 5: Best hyperparameters found (5-fold stratified Grid Search)

Model	Best Parameters	Best CV Accuracy
Bagging	<code>n_estimators=20, max_samples=1.0, max_features=1.0</code>	94.19%
AdaBoost	<code>n_estimators=50, learning_rate=0.5</code>	96.01%
Gradient Boosting	<code>n_estimators=50, learning_rate=0.2, max_depth=1</code>	94.93%
Stacked Ensemble	meta-learner $C=10.0$	95.64%

11 Results

Table 6: Test-set performance comparison ($n = 70$)

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Bagging	88.57%	84.62%	64.71%	73.33%	0.9445
AdaBoost	95.71%	100.00%	82.35%	90.32%	0.9756
Gradient Boosting	94.29%	100.00%	76.47%	86.67%	0.9834
Stacked Ensemble	95.71%	100.00%	82.35%	90.32%	0.9945

12 Result Visualization

1. Correlation Matrix

Not generated by the current script (see Section 4).

2. Class Distribution

Not plotted by the current script, but the underlying numbers are in Section 4's "Class distribution before and after outlier removal" table. A simple `sns.countplot(y)` before/after outlier removal would visualize the imbalance directly if added.

3. Confusion Matrix

Table 7: Raw confusion-matrix counts on the held-out test set

Model	TN (B→B)	FP (B→M)	FN (M→B)	TP (M→M)
Bagging	51	2	6	11
AdaBoost	53	0	3	14
Gradient Boosting	53	0	4	13
Stacked Ensemble	53	0	3	14

Inference: the current script prints these as raw arrays rather than plotted heatmaps; a `seaborn.heatmap` per model (as used in Experiments 3 and 4) would satisfy this visualization requirement directly. Numerically, Bagging is the clear outlier – its 6 false negatives (missed malignant cases) are double any other model, while AdaBoost and the Stacked Ensemble share an identical, best-in-class confusion matrix (only 3 false negatives, 0 false positives).

4. ROC Curve

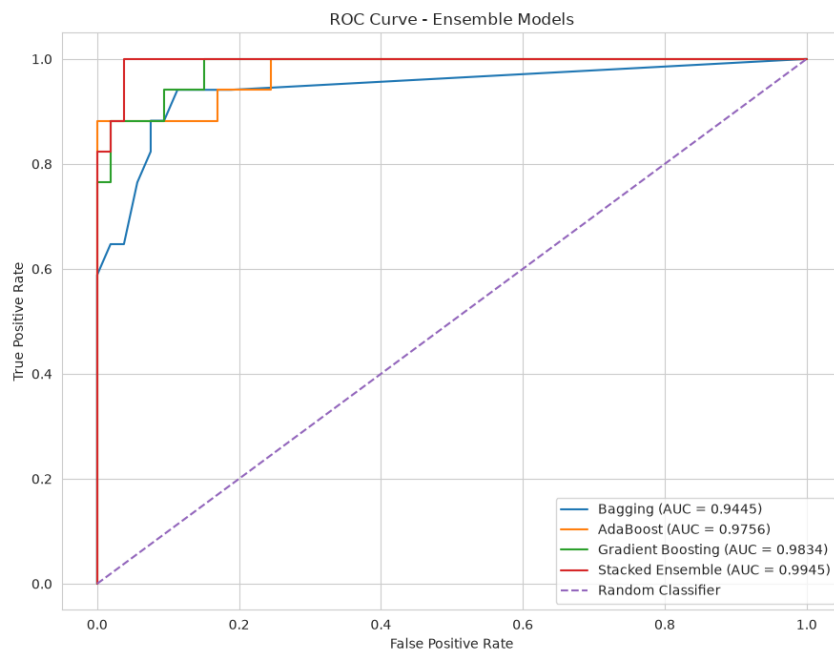


Figure 1: ROC curves for all four ensemble models with AUC values (as generated by `exp7.py`, saved to `plots/ensemble_roc_curve.png`).

Inference: the Stacked Ensemble (AUC = 0.9945) and Gradient Boosting (AUC = 0.9834) sit closest to the top-left corner, indicating the best class separability at any threshold; AdaBoost (AUC = 0.9756) is close behind, while Bagging (AUC = 0.9445) is visibly the least sharp curve of the four – consistent with it having the weakest recall in the Results table.

5. Precision-Recall Curve

Not generated by the current script. Given the class imbalance introduced by outlier removal (Section 4), a Precision-Recall curve would likely be more informative than the ROC curve for comparing these models on the minority (malignant) class, and is recommended as an addition.

6. Learning Curve

Not generated by the current script.

7. Feature Importance

Not generated by the current script, though it is directly available for the three tree-based ensembles (`.feature_importances_` on the fitted Bagging, AdaBoost, and Gradient Boosting estimators) and would help identify which of the 30 features drive the malignant/benign split most strongly – likely candidates based on Section 4 include the "worst" and "mean" size/concavity features.

13 Discussion

Bagging reduces variance. Averaging predictions across independent bootstrap-trained trees smooths out any single tree's overfitting. The hyperparameter grid (Section 8's underlying search) shows accuracy is fairly stable (90.9%–94.2%) across most configurations and improves as `max_samples`/`max_features` approach 1.0 with more estimators – consistent with variance reduction via averaging. However, on the test set Bagging is clearly the weakest model (88.57% accuracy, 64.71% recall): variance reduction alone cannot compensate for the bias introduced by training on only 83 malignant examples (Section 4), and this directly explains its 6 false negatives in the confusion matrix.

Boosting reduces bias. AdaBoost and Gradient Boosting train sequentially, each new learner explicitly correcting the previous ensemble's errors. Both comfortably outperform Bagging (94.29%–95.71% accuracy vs. 88.57%) and achieve perfect precision (100%, zero false positives), at the cost of still missing 3–4 malignant cases – evidence that sequential bias correction works, but cannot fully overcome the imbalanced training signal left after outlier removal.

Stacking benefits from heterogeneous base learners. SVM, Naïve Bayes, and a Decision Tree each make different kinds of errors; the Logistic Regression meta-learner learns how to weight their combined outputs. This shows up as the single best AUC (0.9945) among all four models, even though its hard-threshold predictions tie exactly with AdaBoost – the meta-learner produces better-ranked probability estimates without necessarily changing the default-threshold decision.

Overall: the results align with ensemble theory – Bagging (variance control) plateaus at a bias-limited ceiling, Boosting (bias control) climbs steadily with more rounds/higher learning rate until it too plateaus against the same imbalanced data, and Stacking’s combiner-based approach yields the most discriminative probability outputs of the four.

14 Conclusion

Bagging, Boosting (AdaBoost and Gradient Boosting), and a Stacked Ensemble were implemented and evaluated on the WDBC breast cancer classification task. **AdaBoost** achieved the best raw accuracy and F1-score (95.71%, 90.32%), tying exactly with the **Stacked Ensemble** on every hard classification metric, while the Stacked Ensemble’s superior AUC (0.9945) makes it the more robust choice if the decision threshold is later tuned to prioritize recall on malignant cases. **Bagging** was clearly the weakest of the four, underscoring that variance reduction alone is insufficient when the underlying training data is class-imbalanced – itself a consequence of the aggressive IQR-based outlier removal disproportionately discarding malignant samples (60.8% removed vs. 26.3% for benign). This experiment demonstrates concretely how Bagging targets variance, Boosting targets bias, and Stacking exploits model diversity, and highlights that preprocessing choices (here, outlier removal) can materially shape which ensemble strategy appears “best.”

15 References

- [1] L. Breiman, “Bagging predictors,” *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.
- [2] Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
- [3] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [4] D. H. Wolpert, “Stacked generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
- [5] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [6] W. H. Wolberg, W. N. Street, and O. L. Mangasarian, “Breast Cancer Wisconsin (Diagnostic) Data Set,” UCI Machine Learning Repository, 1995. [Online]. Available: [https://archive.ics.uci.edu/ml/datasets/breast+cancer+wisconsin+\(diagnostic\)](https://archive.ics.uci.edu/ml/datasets/breast+cancer+wisconsin+(diagnostic))